Grundlagen Relationaler Datenbanken

Ein Einsteiger-Tutorial

Was lernst du in diesem Tutorial?

- Normalformen (1NF bis 4NF) verstehen und anwenden
- Kardinalitäten in Tabellen-Beziehungen erklären
- Grundbefehle in SQL: CREATE TABLE, SELECT, INSERT INTO, UPDATE

Autor: Sebastian Meisel | Team Viona Lernbegleitung
Institut für Berufliche Bildung GmbH (IBB)
Stand: 2026 | Für IT-Umschulungen

Hinweis gemäß EU-AI-Act (Artikel 50): Dieses Lernmaterial wurde mit Unterstützung eines KI-Systems erstellt und redaktionell durch Sebastian Meisel geprüft. Es dient ausschließlich Bildungszwecken.

Kapitel 1: Was ist eine relationale Datenbank?

Eine Datenbank ist wie ein digitales Notizbuch - nur viel größer und besser organisiert. Statt Zettel zu schreiben, speichern wir Daten in Tabellen.

> Stell dir vor, du hast eine Excel-Tabelle mit all deinen Kontakten. Eine Datenbank macht genau das - aber für Millionen von Einträgen gleichzeitig, und mehrere Menschen können gleichzeitig damit arbeiten.

1.1 Tabellen - Das Herzstuck

In einer relationalen Datenbank sind alle Daten in Tabellen gespeichert. Jede Tabelle hat:

- Spalten (auch: Felder oder Attribute) - was wird gespeichert?
- Zeilen (auch: Datensätze oder Tupel) - ein konkreter Eintrag
- Einen Primärschlüssel (PK) - eine eindeutige ID für jede Zeile

Beispiel: Tabelle SCHUELER

schueler_id (PK)	vorname	nachname	kurs
1	Anna	Müller	Fachinformatiker
2	Ben	Weber	IT-Kaufmann
3	Cara	Braun	Fachinformatiker

Der Primärschlüssel (PK) stellt sicher, dass jede Zeile eindeutig identifiziert werden kann. Kein Wert darf doppelt vorkommen!

Kapitel 2: Normalformen (1NF bis 4NF)

Normalformen sind Regeln, nach denen wir Tabellen sauber strukturieren. Ziel: keine Wiederholungen, keine Widersprüche, keine Fehler.



2.1 Erste Normalform (1NF)

> Regel: Jede Tabellenzelle darf nur EINEN Wert enthalten. Keine Listen in einer Zelle!

PROBLEM - so ist es falsch:

schueler_id	name	kurse
1	Anna	Python, Excel, SQL
2	Ben	SQL, Excel

LOESUNG - so ist es richtig (1NF):

schueler_id	name	kurs
1	Anna	Python
1	Anna	Excel
1	Anna	SQL
2	Ben	SQL
2	Ben	Excel

Jetzt steht in jeder Zelle genau ein Wert. Das ist die 1. Normalform.

2.2 Zweite Normalform (2NF)

Regel: Jede Nicht-Schlüssel-Spalte muss vom GANZEN Primärschlüssel abhängig sein. Keine halben Abhängigkeiten!

Was bedeutet das? Wenn der Primärschlüssel aus mehreren Spalten besteht (zusammengesetzter PK), muss jede andere Spalte von ALLEN PK-Spalten abhängen - nicht nur von einer.

PROBLEM: Tabelle mit zusammengesetztem PK:

schueler_id (PK)	kurs_id (PK)	schüler_name	kursname
1	101	Anna	Python
1	102	Anna	Excel
2	101	Ben	Python

Hier hängt 'schueler_name' nur von schueler_id ab, und 'kursname' nur von kurs_id. Das verstößt gegen 2NF!

LÖSUNG - aufteilen in zwei Tabellen:

Tabelle SCHUELER:

schueler_id (PK)	schüler_name
1	Anna
2	Ben
3	Cara

Tabelle KURS:

kurs_id (PK)	kursname
101	Python
102	Excel
103	SQL

2.3 Dritte Normalform (3NF)

Regel: Keine Nicht-Schlüssel-Spalte darf von einer anderen Nicht-Schlüssel-Spalte abhängen (keine transitiven Abhängigkeiten).

PROBLEM:

schueler_id (PK)	name	plz	stadt
1	Anna	80331	München
2	Ben	20095	Hamburg

schueler_id (PK)	name	plz	stadt
3	Cara	80331	München

Die Stadt hängt von der PLZ ab - nicht direkt vom Primärschlüssel. Das ist eine transitive Abhängigkeit!

LÖSUNG - Tabelle aufteilen:

Tabelle SCHUELER:

schueler_id (PK)	name	plz
1	Anna	80331
2	Ben	20095
3	Cara	80331

Tabelle ORT:

plz (PK)	stadt
80331	München
20095	Hamburg

2.4 Vierte Normalform (4NF)

Regel: Eine Tabelle darf keine mehrwertigen Abhängigkeiten enthalten, die unabhängig voneinander sind.

Die 4NF ist eine Erweiterung für fortgeschrittene Fälle. Sie tritt auf, wenn eine Entität zwei oder mehr voneinander unabhängige Mehrfach-Beziehungen hat.

PROBLEM: Ein Lehrer unterrichtet mehrere Kurse UND spricht mehrere Sprachen:

lehrer_id	kurs	sprache
1	Python	Deutsch
1	Python	Englisch
1	SQL	Deutsch
1	SQL	Englisch

Kurse und Sprachen sind unabhängig voneinander - trotzdem werden sie kombiniert. Das führt zu vielen Duplikaten!

LLSUNG - in zwei Tabellen trennen:

Tabelle LEHRER_KURS:

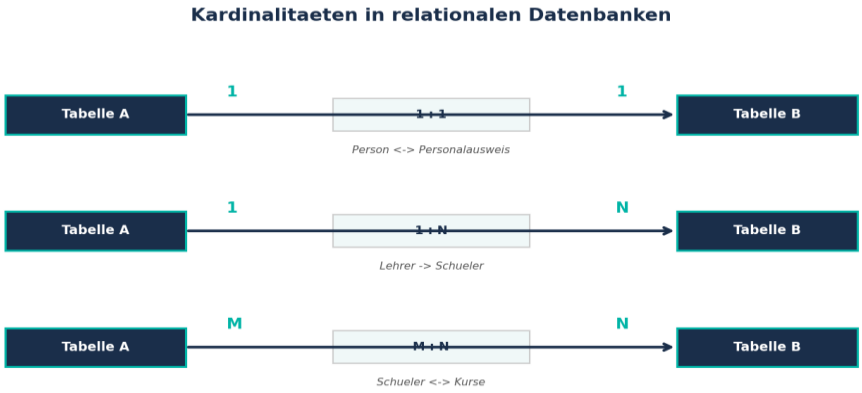
lehrer_id	kurs
1	Python
1	SQL

Tabelle LEHRER_SPRACHE:

lehrer_id	sprache
1	Deutsch
1	Englisch

Kapitel 3: Kardinalitäten

Kardinalitäten beschreiben, wie viele Datensätze einer Tabelle mit wie vielen Datensätzen einer anderen Tabelle verknüpft sein können.



3.1 Eins-zu-Eins (1:1)

Ein Datensatz in Tabelle A gehört genau zu einem Datensatz in Tabelle B - und umgekehrt.

Beispiel: Jede Person hat genau einen Personalausweis.

person_id (PK)	name
1	Anna
2	Ben

ausweis_id (PK)	person_id (FK)	gültig_bis
A001	1	2030-01-15
A002	2	2028-06-22

FK = Fremdschlüssel (Foreign Key). Er verweist auf den Primärschlüssel einer anderen Tabelle.

3.2 Eins-zu-Viele (1:N)

Ein Datensatz in Tabelle A kann mit VIELEN Datensätzen in Tabelle B verknüpft sein. Aber jeder Datensatz in B gehört nur zu einem in A.

Beispiel: Ein Lehrer hat viele Schüler. Jeder Schüler hat aber nur einen Lehrer.

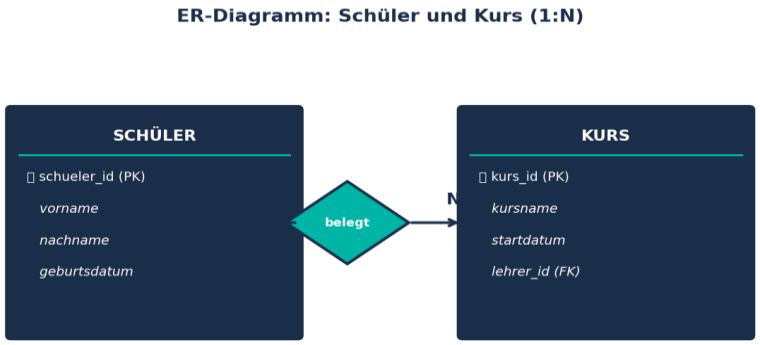


Tabelle LEHRER:

lehrer_id (PK)	name
1	Fr. Schmidt
2	Hr. Müller

Tabelle SCHUELER:

schueler_id (PK)	name	lehrer_id (FK)
101	Anna	1
102	Ben	1
103	Cara	2

Anna und Ben gehören zu Fr. Schmidt (lehrer_id=1). Cara gehört zu Hr. Müller.

3.3 Viele-zu-Viele (M:N)

Viele Datensätze in A können mit vielen Datensätzen in B verknüpft sein. Dafür brauchen wir eine ZWISCHENTABELLE.

Beispiel: Schüler können viele Kurse belegen. Jeder Kurs hat viele Schüler.

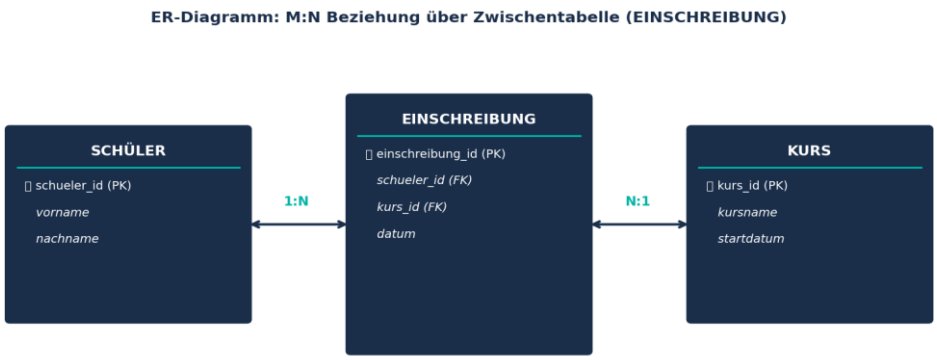


Tabelle SCHUELER:

schueler_id (PK)	name
1	Anna
2	Ben
3	Cara

Tabelle KURS:

kurs_id (PK)	kursname
101	Python
102	SQL
103	Excel

Zwischentabelle EINSCHREIBUNG:

einschreibung_id (PK)	schueler_id (FK)	kurs_id (FK)	datum
1	1	101	2026-01-10
2	1	102	2026-01-10
3	2	101	2026-01-12
4	3	103	2026-01-15

Anna belegt Python und SQL. Ben belegt Python. Cara belegt Excel.

Kapitel 4: SQL-Grundbefehle

SQL steht für Structured Query Language - die Sprache, mit der wir mit Datenbanken sprechen. Hier lernst du die vier wichtigsten Befehle.

4.1 CREATE TABLE - Tabelle erstellen

CREATE TABLE legt eine neue Tabelle in der Datenbank an. Du bestimmst, welche Spalten es gibt und welchen Datentyp sie haben.

Syntax:

```
CREATE TABLE tabellenname (
    spaltenname1  DATENTYP  OPTIONEN,
    spaltenname2  DATENTYP  OPTIONEN,
    ...
);
```

Konkretes Beispiel - Tabelle SCHUELER erstellen:

```
CREATE TABLE schüler (
    schüler_id   INT           PRIMARY KEY,
    vorname       VARCHAR(50)   NOT NULL,
    nachname      VARCHAR(50)   NOT NULL,
    geburtsdatum  DATE
);
```

Was bedeuten die Datentypen?

Datentyp	Bedeutung	Beispiel
INT	Ganze Zahl	42, 1000, -5
VARCHAR(n)	Text bis n Zeichen	'Anna', 'Berlin'
DATE	Datum	'2025-01-15'
DECIMAL(p,s)	Kommazahl	3.14, 99.99
BOOLEAN	Wahr/Falsch	TRUE, FALSE

Wichtige Optionen:

- PRIMARY KEY - eindeutiger Identifikator der Zeile
- NOT NULL - dieses Feld muss ausgefüllt sein
- UNIQUE - kein Wert darf zweimal vorkommen
- DEFAULT wert - Standardwert, wenn nichts eingegeben wird

4.2 INSERT INTO - Daten einfuegen

INSERT INTO fuegt neue Zeilen (Datensätze) in eine Tabelle ein.

Syntax:

```
INSERT INTO tabellenname (spalte1, spalte2, ...)
VALUES (wert1, wert2, ...);
```

Beispiel - Schüler einfuegen:

```
INSERT INTO schüler (schüler_id, vorname, nachname, geburtsdatum)
VALUES (1, 'Anna', 'Müller', '1998-03-15');

INSERT INTO schüler (schüler_id, vorname, nachname, geburtsdatum)
VALUES (2, 'Ben', 'Weber', '2001-07-22');
```

Das Ergebnis in der Tabelle:

schueler_id	vorname	nachname	geburtsdatum
1	Anna	Müller	1998-03-15
2	Ben	Weber	2001-07-22

Tipp: Textwerte (Strings) werden in einfache Anführungszeichen gesetzt: 'Anna'. Zahlen und Daten brauchen das nicht immer, aber Strings immer!

4.3 SELECT - Daten abfragen

SELECT ist der häufigste Befehl in SQL. Er liest Daten aus einer oder mehreren Tabellen.

Syntax:

```
SELECT spalte1, spalte2, ...
FROM tabellenname
WHERE bedingung;
```

Alle Daten aus einer Tabelle anzeigen:

```
SELECT *
FROM schüler;
```

Das * bedeutet: alle Spalten. Das wäre das Ergebnis:

schueler_id	vorname	nachname	geburtsdatum
1	Anna	Müller	1998-03-15
2	Ben	Weber	2001-07-22
3	Cara	Braun	2000-11-08

Nur bestimmte Spalten anzeigen:

```
SELECT vorname, nachname
FROM schüler;
```

Mit WHERE filtern - nur Schüler mit einer bestimmten ID:

```
SELECT *
FROM schüler
WHERE schüler_id = 1;
```

Das Ergebnis:

schueler_id	vorname	nachname	geburtsdatum
1	Anna	Müller	1998-03-15

Weitere nützliche WHERE-Bedingungen:

Operator	Bedeutung	Beispiel
=	gleich	WHERE id = 5
<> oder !=	ungleich	WHERE name <> 'Ben'
>	größer als	WHERE schüler_id > 2
LIKE	Textmuster	WHERE name LIKE 'A%'
AND	und (beide Bedingungen)	WHERE id > 1 AND id < 4
OR	oder (mind. eine Bedingung)	WHERE name = 'Anna' OR name = 'Ben'

4.4 UPDATE - Daten ändern

UPDATE ändert vorhandene Datensätze in einer Tabelle. WICHTIG: Vergiss niemals das WHERE - sonst werden ALLE Zeilen geändert!

Syntax:

```
UPDATE tabellenname
SET spalte1 = neuer_wert, spalte2 = neuer_wert
WHERE bedingung;
```

Beispiel - Nachname von Schüler 1 ändern:

```
UPDATE schüler
SET nachname = 'Meier'
WHERE schüler_id = 1;
```

Vorher:

schueler_id	vorname	nachname
1	Anna	Müller
2	Ben	Weber

Nachher:

schueler_id	vorname	nachname
1	Anna	Meier
2	Ben	Weber

Achtung! Das passiert ohne WHERE:

```
UPDATE schüler SET nachname = 'Meier';
```

=> ALLE Schüler in der Tabelle würden 'Meier' heißen! Immer WHERE verwenden!

Kapitel 5: Übungen

Teste dein Wissen! Versuche die Aufgaben erst selbst zu lösen, bevor du die Lösungen schaust.

Übung 1 - Normalformen

Schau dir diese Tabelle an. Welche Normalform verletzt sie? Wie würdest du sie korrigieren?

bestell_id	kunde	produkte	preise
1	Anna	Laptop, Maus	999, 29
2	Ben	Tastatur	49

Deine Antwort: _____

_____

Übung 2 - Kardinalitäten

Welche Kardinalität liegt vor?

- Eine Stadt hat viele Einwohner. Ein Einwohner lebt in einer Stadt. => ___:___
- Ein Autor schreibt viele Buecher. Ein Buch hat genau einen Autor. => ___:___
- Studierende können viele Vorlesungen besuchen. Vorlesungen haben viele Studierende. => ___:___

Übung 3 - SQL

Erstelle die SQL-Befehle für folgende Aufgaben:

Aufgabe 3a: Erstelle eine Tabelle 'produkt' mit den Spalten: produkt_id (PK), name, preis (Dezimalzahl), auf_lager (Wahr/Falsch).

```
-- Deine Lösung hier:
```

Aufgabe 3b: Fuege zwei Produkte ein: 'Laptop' für 999.99 EUR (auf Lager) und 'Maus' für 29.50 EUR (nicht auf Lager).

```
-- Deine Lösung hier:
```

Aufgabe 3c: Zeige nur Produkte an, die auf Lager sind.

```
-- Deine Lösung hier:
```

Aufgabe 3d: Aendere den Preis des Laptops auf 899.99 EUR.

```
-- Deine Lösung hier:
```

Kapitel 6: Lösungen

Lösung zu Übung 1

Die Tabelle verletzt die ERSTE Normalform (1NF). Die Spalten 'produkte' und 'preise' enthalten jeweils mehrere Werte.

Korrektur (1NF):

bestell_id	kunde	produkt	preis
1	Anna	Laptop	999
1	Anna	Maus	29
2	Ben	Tastatur	49

Für 2NF/3NF würde man zusätzlich Kunden- und Produkttabellen trennen.

Lösung zu Übung 2

- Stadt - Einwohner: 1:N
- Autor - Bücher: 1:N
- Studierende - Vorlesungen: M:N (braucht Zwischentabelle!)

Lösung zu Übung 3

3a - CREATE TABLE:

```
CREATE TABLE produkt (
    produkt_id  INT             PRIMARY KEY,
    name        VARCHAR(100)    NOT NULL,
    preis       DECIMAL(10, 2)  NOT NULL,
    auf_lager   BOOLEAN         DEFAULT TRUE
);
```

3b - INSERT INTO:

```
INSERT INTO produkt (produkt_id, name, preis, auf_lager)
VALUES (1, 'Laptop', 999.99, TRUE);

INSERT INTO produkt (produkt_id, name, preis, auf_lager)
VALUES (2, 'Maus', 29.50, FALSE);
```

3c - SELECT mit WHERE:

```
SELECT *
FROM produkt
WHERE auf_lager = TRUE;
```

3d - UPDATE:

```
UPDATE produkt
SET preis = 899.99
WHERE produkt_id = 1;
```

Glossar

Begriff	Erklärung
Datenbank	Organisiertes System zum Speichern und Verwalten großer Datenmengen
Tabelle	Struktur aus Zeilen und Spalten zur Datenspeicherung
Zeile / Tupel	Ein einzelner Datensatz in einer Tabelle
Spalte / Attribut	Eine Eigenschaft, die für alle Zeilen gespeichert wird
Primärschlüssel (PK)	Eindeutiger Identifikator für jede Zeile einer Tabelle
Fremdschlüssel (FK)	Verweis auf den PK einer anderen Tabelle
Normalform	Regel zur sauberen Strukturierung von Datenbanktabellen
Kardinalität	Beschreibt, wie viele Datensätze zueinander in Beziehung stehen
1:N	Ein Datensatz in A gehört zu vielen in B
M:N	Viele Datensätze in A gehören zu vielen in B (Zwischentabelle nötig)
SQL	Structured Query Language - Sprache für Datenbankbefehle
CREATE TABLE	SQL-Befehl: neue Tabelle anlegen
INSERT INTO	SQL-Befehl: Datensatz einfügen
SELECT	SQL-Befehl: Daten abfragen/lesen
UPDATE	SQL-Befehl: vorhandene Daten ändern
WHERE	Bedingung in SQL, um Daten zu filtern
VARCHAR(n)	Datentyp für Text bis n Zeichen
INT	Datentyp für ganze Zahlen
DECIMAL(p,s)	Datentyp für Kommazahlen
Zwischentabelle	Tabelle zur Auflösung einer M:N-Beziehung

Hinweis gemäß EU-AI-Act (Artikel 50): Dieses Lernmaterial wurde mit Unterstützung eines KI-Systems (Langdock Auto) erstellt und inhaltlich durch Sebastian Meisel, Team Viona Lernbegleitung, geprüft und freigegeben. Es dient ausschließlich Bildungszwecken im Rahmen der IT-Umschulung am IBB. Stand: August 2026.